

Universidad de Costa Rica

Facultad de Ingeniería

Escuela de Ciencias de la Computación e Informática

Bases de Datos Avanzadas

Laboratorio Cassandra



Profesor

Luis Gustavo Esquivel

Estudiante

Brandon Trigueros Lara C17899

Semestre I

2025

1. Configurar repositorio oficial de Docker

```
sudo dnf -y install dnf-plugins-core
```

De serie Fedora puede no traer habilitado el subcomando `dnf config-manager`. Al instalar `dnf-plugins-core`, disponemos de esa y otras utilidades avanzadas para gestionar repositorios, módulos y más.

```
sudo dnf config-manager addrepo  
--from-repofile=https://download.docker.com/linux/fedora/docker-ce.repo
```

Tras esto, `dnf` sabrá dónde buscar e instalar los paquetes de Docker CE (Community Edition) directamente desde los servidores de Docker, manteniéndolos actualizados y firmados.

```
sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin -y
```

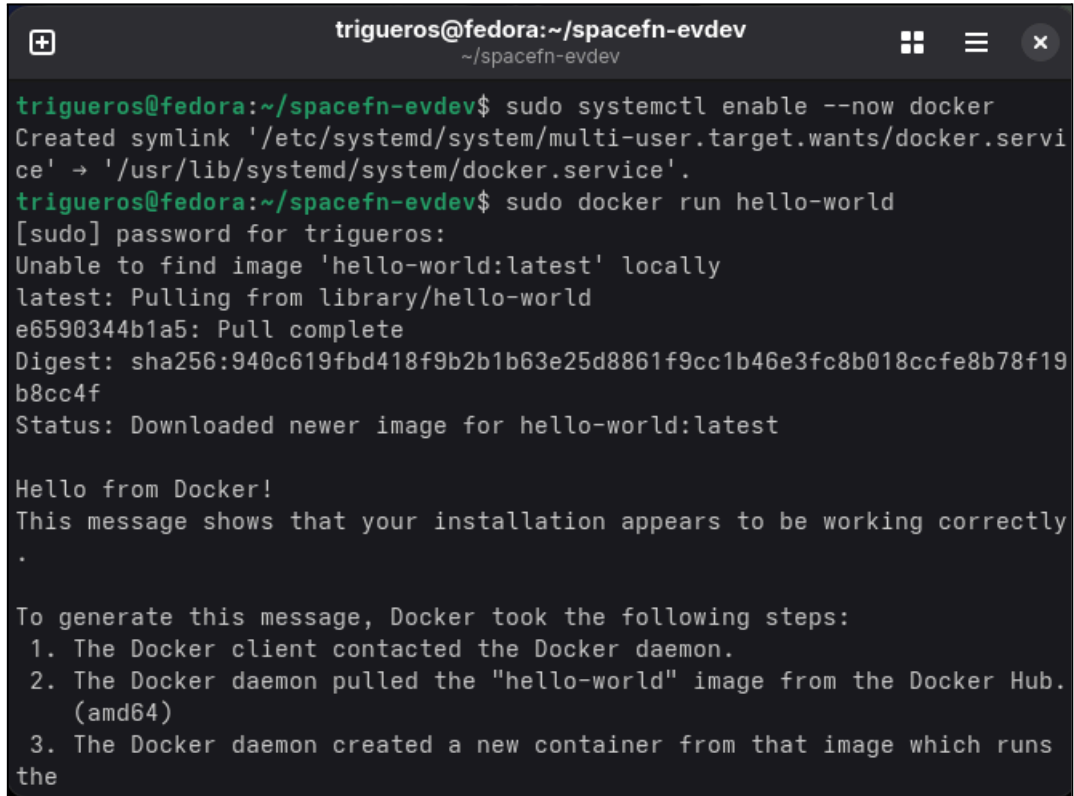
Este comando instala Docker y sus componentes sin iniciarlo automáticamente:

```
trigueros@fedora:~/spacefn-evdev — sudo dnf install docker-c...  
~/spacefn-evdev  
docker-ce-cli           x86_64 1:28.3.0-1.fc42      docker-c 35.3 MiB  
docker-compose-plugin   x86_64 2.37.3-1.fc42            docker-c 72.5 MiB  
Installing dependencies:  
libcgroupp              x86_64 3.0-8.fc42               fedora   157.7 KiB  
slirp4netns              x86_64 1.3.1-2.fc42             fedora   89.3 KiB  
Installing weak dependencies:  
docker-ce-rootless-extras x86_64 28.3.0-1.fc42            docker-c 11.2 MiB  
  
Transaction Summary:  
Installing:          8 packages  
  
Total size of inbound packages is 107 MiB. Need to download 107 MiB.  
After this operation, 434 MiB extra will be used (install 434 MiB, remove  
0 B).  
[1/8] containerd.io-0:1.7.27-3.1 16% | 3.4 MiB/s | 7.1 MiB | 00m10s  
[1/8] docker-ce-cli-1:28.3.0-1.f 100% | 1.9 MiB/s | 8.5 MiB | 00m05s  
[2/8] docker-ce-3:28.3.0-1.fc42. 100% | 3.0 MiB/s | 20.3 MiB | 00m07s  
[3/8] containerd.io-0:1.7.27-3.1 46% | 4.2 MiB/s | 20.4 MiB | 00m05s  
[4/8] docker-buildx-plugin-0:0.2 5% | 503.2 KiB/s | 945.5 KiB | 00m31s  
[5/8] docker-compose-plugin-0:2. 0% | 3.0 B/s | 0.0 B | 60d03h  
-----  
[2/8] Total 46% | 9.4 MiB/s | 50.1 MiB | 00m06s
```

```
sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin -y
```

```
sudo docker run hello-world
```

Configura Docker para que arranque al inicio y lo pone en marcha inmediatamente a la vez que verifica que la instalación fue correcta

A terminal window titled 'trigueros@fedora:~/spacefn-evdev' with a dark background. It shows the execution of 'sudo systemctl enable --now docker', which creates a symlink. Then 'sudo docker run hello-world' is run, prompting for a password. The output shows the 'hello-world:latest' image being pulled from Docker Hub, with its digest and status. Finally, the container runs and outputs 'Hello from Docker!' and a confirmation message.

```
trigueros@fedora:~/spacefn-evdev$ sudo systemctl enable --now docker
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service' → '/usr/lib/systemd/system/docker.service'.
trigueros@fedora:~/spacefn-evdev$ sudo docker run hello-world
[sudo] password for trigueros:
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
e6590344b1a5: Pull complete
Digest: sha256:940c619fbd418f9b2b1b63e25d8861f9cc1b46e3fc8b018ccfe8b78f19b8cc4f
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly
.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
```

```
sudo systemctl status docker
```

Este comando revisa el estado del servicio de Docker para verificar que se está corriendo.

```
trigueros@fedora:~/spacefn-evdev — sudo systemctl status do...
~/spacefn-evdev

trigueros@fedora:~/spacefn-evdev$ sudo systemctl status docker
[sudo] password for trigueros:
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; pr>
   Drop-In: /usr/lib/systemd/system/service.d
           └─10-timeout-abort.conf
   Active: active (running) since Tue 2025-07-01 15:19:07 CST; 31min a>
   Invocation: 277fea3ae5c147e6a786e536461b6bbf
   TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 9962 (dockerd)
   Tasks: 17
   Memory: 32.9M (peak: 38.8M)
   CPU: 697ms
   CGroup: /system.slice/docker.service
           └─9962 /usr/bin/dockerd -H fd:// --containerd=/run/containe>

Jul 01 15:19:06 fedora dockerd[9962]: time="2025-07-01T15:19:06.27645952>
Jul 01 15:19:06 fedora dockerd[9962]: time="2025-07-01T15:19:06.29845202>
Jul 01 15:19:07 fedora dockerd[9962]: time="2025-07-01T15:19:07.11541429>
Jul 01 15:19:07 fedora dockerd[9962]: time="2025-07-01T15:19:07.14102103>
```

2. Instalación de kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

Obtiene la versión estable más reciente directamente desde los servidores de Kubernetes:

```
trigueros@fedora:~/spacefn-evdev
~/spacefn-evdev

38 57.3M 38 21.9M 0 0 1784k 0 0:00:32 0:00:12 0:00:20
41 57.3M 41 23.7M 0 0 1795k 0 0:00:32 0:00:13 0:00:19
44 57.3M 44 25.5M 0 0 1794k 0 0:00:32 0:00:14 0:00:18
47 57.3M 47 27.3M 0 0 1795k 0 0:00:32 0:00:15 0:00:17
50 57.3M 50 29.2M 0 0 1804k 0 0:00:32 0:00:16 0:00:16
54 57.3M 54 31.2M 0 0 1821k 0 0:00:32 0:00:17 0:00:15
57 57.3M 57 33.0M 0 0 1820k 0 0:00:32 0:00:18 0:00:14
59 57.3M 59 34.3M 0 0 1792k 0 0:00:32 0:00:19 0:00:13
62 57.3M 62 36.0M 0 0 1794k 0 0:00:32 0:00:20 0:00:12
65 57.3M 65 37.8M 0 0 1794k 0 0:00:32 0:00:21 0:00:11
69 57.3M 69 39.7M 0 0 1805k 0 0:00:32 0:00:22 0:00:10
72 57.3M 72 41.7M 0 0 1812k 0 0:00:32 0:00:23 0:00:09
75 57.3M 75 43.5M 0 0 1814k 0 0:00:32 0:00:24 0:00:08
79 57.3M 79 45.4M 0 0 1819k 0 0:00:32 0:00:25 0:00:07
82 57.3M 82 47.3M 0 0 1823k 0 0:00:32 0:00:26 0:00:06
85 57.3M 85 49.1M 0 0 1826k 0 0:00:32 0:00:27 0:00:05
89 57.3M 89 51.0M 0 0 1828k 0 0:00:32 0:00:28 0:00:04
91 57.3M 91 52.6M 0 0 1820k 0 0:00:32 0:00:29 0:00:03
96 57.3M 96 55.0M 0 0 1844k 0 0:00:31 0:00:30 0:00:01
100 57.3M 100 57.3M 0 0 1864k 0 0:00:31 0:00:31 --:--:--
2087k
trigueros@fedora:~/spacefn-evdev$
```

```
chmod +x kubect1
```

```
sudo install -o root -g root -m 0755 kubect1 /usr/local/bin/kubect1
```

Hace que el binario sea ejecutable e instala **kubect1** en **/usr/local/bin** para que esté disponible globalmente:

```
kubect1 version --client
```

Verificamos que esté el cliente instalado consultando su versión:

```
trigueros@fedora:~/spacefn-evdev
~/spacefn-evdev

57 57.3M 57 33.0M 0 0 1820k 0 0:00:32 0:00:18 0:00:14
59 57.3M 59 34.3M 0 0 1792k 0 0:00:32 0:00:19 0:00:13
62 57.3M 62 36.0M 0 0 1794k 0 0:00:32 0:00:20 0:00:12
65 57.3M 65 37.8M 0 0 1794k 0 0:00:32 0:00:21 0:00:11
69 57.3M 69 39.7M 0 0 1805k 0 0:00:32 0:00:22 0:00:10
72 57.3M 72 41.7M 0 0 1812k 0 0:00:32 0:00:23 0:00:09
75 57.3M 75 43.5M 0 0 1814k 0 0:00:32 0:00:24 0:00:08
79 57.3M 79 45.4M 0 0 1819k 0 0:00:32 0:00:25 0:00:07
82 57.3M 82 47.3M 0 0 1823k 0 0:00:32 0:00:26 0:00:06
85 57.3M 85 49.1M 0 0 1826k 0 0:00:32 0:00:27 0:00:05
89 57.3M 89 51.0M 0 0 1828k 0 0:00:32 0:00:28 0:00:04
91 57.3M 91 52.6M 0 0 1820k 0 0:00:32 0:00:29 0:00:03
96 57.3M 96 55.0M 0 0 1844k 0 0:00:31 0:00:30 0:00:01
100 57.3M 100 57.3M 0 0 1864k 0 0:00:31 0:00:31 --:--:--
2087k
trigueros@fedora:~/spacefn-evdev$ chmod +x kubectl
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
[sudo] password for trigueros:
trigueros@fedora:~/spacefn-evdev$ kubectl version --client
Client Version: v1.33.2
Kustomize Version: v5.6.0
trigueros@fedora:~/spacefn-evdev$
```

3. Instalación de k3d

```
wget -q -O - https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
```

Descarga y ejecuta el script oficial de instalación de k3d

```
trigueros@fedora:~/spacefn-evdev
~/spacefn-evdev

trigueros@fedora:~/spacefn-evdev$ wget -q -O - https://raw.githubusercontent.com/k3d-io/k3d/main/install.sh | bash
Preparing to install k3d into /usr/local/bin
[sudo] password for trigueros:
Sorry, try again.
[sudo] password for trigueros:
k3d installed into /usr/local/bin/k3d
Run 'k3d --help' to see what you can do with it.
trigueros@fedora:~/spacefn-evdev$
```

```
k3d version
```

Muestra la versión instalada de k3d y de k3s integrado

```
trigueros@fedora:~/spacefn-evdev$ k3d version
k3d version v5.8.3
k3s version v1.31.5-k3s1 (default)
trigueros@fedora:~/spacefn-evdev$
```

4. Creación del clúster con k3d

```
sudo k3d cluster create multi-node-cluster \
  --agents 6 \
  --k3s-arg "--node-name=nodea1@agent:0" \
  --k3s-arg "--node-name=nodea2@agent:1" \
  --k3s-arg "--node-name=nodea3@agent:2" \
  --k3s-arg "--node-name=nodeb1@agent:3" \
  --k3s-arg "--node-name=nodeb2@agent:4" \
  --k3s-arg "--node-name=nodeb3@agent:5"
```

--agents 6: crea 6 nodos worker (3 para DC1, 3 para DC2).

Cada **--k3s-arg "--node-name=...@agent:n"** asigna un nombre específico a cada contenedor-nodo

```
sudo kubectl cluster-info
```

Verifica que **kubectl** esté apuntando correctamente al clúster local

```
trigueros@fedora:~/spacefn-evdev$ sudo kubectl cluster-info

Broadcast message from systemd-journald@fedora (Tue 2025-07-01 16:33:39 C
ST):

systemd[23595]: Failed to allocate manager object: Too many open files

Kubernetes control plane is running at https://0.0.0.0:38547
CoreDNS is running at https://0.0.0.0:38547/api/v1/namespaces/kube-system
/services/kube-dns:dns/proxy
Metrics-server is running at https://0.0.0.0:38547/api/v1/namespaces/kube
-system/services/https:metrics-server:https/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info
dump'.

Message from syslogd@fedora at Jul  1 16:33:39 ...
systemd[23595]:Failed to allocate manager object: Too many open files
trigueros@fedora:~/spacefn-evdev$
```

```
k3d kubeconfig merge multi-node-cluster \  
  --kubeconfig-merge-default \  
  --kubeconfig-switch-context
```

Fusionar el kubeconfig dentro de tu archivo por defecto (~/.kube/config) usando la opción --kubeconfig-merge-default y cambiar el contexto activo con switch-context

```
sudo kubectl get nodes
```

Muestra los 7 nodos (1 control-plane + 6 agentes) disponibles

```
--kubeconfig-merge-default \  
--kubeconfig-switch-context  
/home/trigueros/.kube/config  
trigueros@fedora:~/spacefn-evdev$ kubectl config current-context  
k3d-multi-node-cluster  
trigueros@fedora:~/spacefn-evdev$ kubectl get nodes  
NAME                                STATUS    ROLES                                AGE    VERSION  
k3d-multi-node-cluster-server-0    Ready    control-plane,master                31m    v1.31.5+k3s1  
nodea1                              Ready    <none>                              30m    v1.31.5+k3s1  
nodea2                              Ready    <none>                              30m    v1.31.5+k3s1  
nodea3                              Ready    <none>                              30m    v1.31.5+k3s1  
nodeb1                              Ready    <none>                              30m    v1.31.5+k3s1  
nodeb2                              Ready    <none>                              30m    v1.31.5+k3s1  
nodeb3                              Ready    <none>                              30m    v1.31.5+k3s1
```

5. Etiquetado de nodos por data center

```
sudo kubectl label nodes nodea1 dc=DC1  
sudo kubectl label nodes nodea2 dc=DC1  
sudo kubectl label nodes nodea3 dc=DC1
```

Asigna la etiqueta **dc=DC1** para asegurar afinidad de scheduling

```
sudo kubectl label nodes nodeb1 dc=DC2  
sudo kubectl label nodes nodeb2 dc=DC2  
sudo kubectl label nodes nodeb3 dc=DC2
```

Asigna la etiqueta **dc=DC2** para el segundo data center

```
sudo kubectl get nodes --show-labels
```


Confirma que cada nodo tenga la etiqueta `dc` correcta:

```
trigueros@fedora:~  
nodea3          Ready    <none>          36m    v  
1.31.5+k3s1     beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-t  
ype=k3s,beta.kubernetes.io/os=linux,dc=DC1,kubernetes.io/arch=amd64,kuber  
netes.io/hostname=nodea3,kubernetes.io/os=linux,node.kubernetes.io/instan  
ce-type=k3s  
nodeb1          Ready    <none>          36m    v  
1.31.5+k3s1     beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-t  
ype=k3s,beta.kubernetes.io/os=linux,dc=DC2,kubernetes.io/arch=amd64,kuber  
netes.io/hostname=nodeb1,kubernetes.io/os=linux,node.kubernetes.io/instan  
ce-type=k3s  
nodeb2          Ready    <none>          36m    v  
1.31.5+k3s1     beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-t  
ype=k3s,beta.kubernetes.io/os=linux,dc=DC2,kubernetes.io/arch=amd64,kuber  
netes.io/hostname=nodeb2,kubernetes.io/os=linux,node.kubernetes.io/instan  
ce-type=k3s  
nodeb3          Ready    <none>          36m    v  
1.31.5+k3s1     beta.kubernetes.io/arch=amd64,beta.kubernetes.io/instance-t  
ype=k3s,beta.kubernetes.io/os=linux,dc=DC2,kubernetes.io/arch=amd64,kuber  
netes.io/hostname=nodeb3,kubernetes.io/os=linux,node.kubernetes.io/instan  
ce-type=k3s  
trigueros@fedora:~$
```

6. Creación del namespace

```
sudo kubectl create namespace c7a
```

Crea el espacio de nombres donde desplegaremos Cassandra y recursos asociados:

```
trigueros@fedora:~$ kubectl create namespace c7a  
namespace/c7a created  
trigueros@fedora:~$
```

7. Clonado del repositorio y descripción de archivos

```
git clone https://github.com/ideagw/multi-dc-c7a-k8s.git  
cd multi-dc-c7a-k8s
```

Obtiene los manifiestos YAML utilizados en el tutorial.

service.yaml

Define dos *headless services* sin `clusterIP` para exponer Cassandra por nombre de DNS

local_pvs.yaml

Provisiona 6 *PersistentVolumes* locales de 10 Gi usando `hostPath` en `/data/cass/...`

statefulset-a.yaml

StatefulSet para DC1 con afinidad `dc=DC1`, una réplica inicial, variables de entorno para seeds y configuración:

statefulset-b.yaml

Mismo esquema que `statefulset-a.yaml`, pero con afinidad `dc=DC2` y variable `CASSANDRA_DC="DC2"`

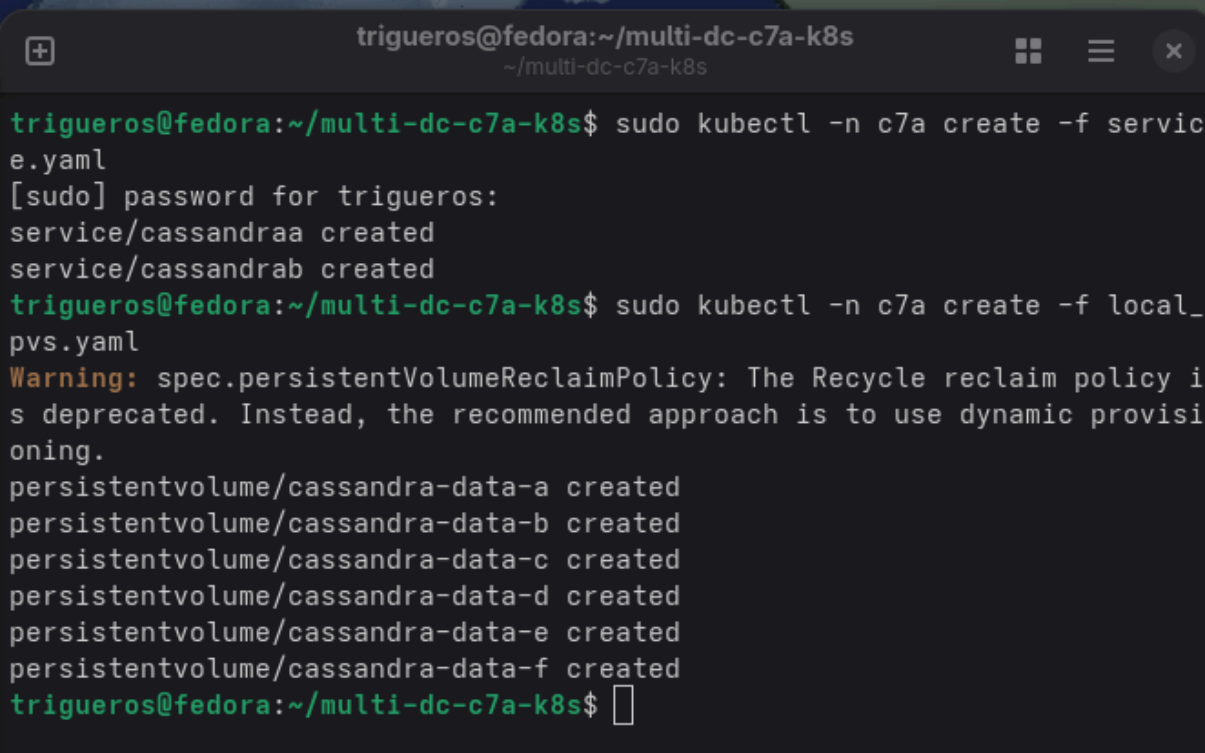
8. Despliegue de servicios y volúmenes

```
sudo kubectl -n c7a create -f service.yaml
```

Crea `cassandraa` y `cassandrab` sin IP de clúster para DNS directo

```
sudo kubectl -n c7a create -f local_pvs.yaml
```

Provisiona seis volúmenes locales de 10 Gi cada uno



```
trigueros@fedora:~/multi-dc-c7a-k8s
~/multi-dc-c7a-k8s

trigueros@fedora:~/multi-dc-c7a-k8s$ sudo kubectl -n c7a create -f service.yaml
[sudo] password for trigueros:
service/cassandraa created
service/cassandrab created
trigueros@fedora:~/multi-dc-c7a-k8s$ sudo kubectl -n c7a create -f local_pvs.yaml
Warning: spec.persistentVolumeReclaimPolicy: The Recycle reclaim policy is deprecated. Instead, the recommended approach is to use dynamic provisioning.
persistentvolume/cassandra-data-a created
persistentvolume/cassandra-data-b created
persistentvolume/cassandra-data-c created
persistentvolume/cassandra-data-d created
persistentvolume/cassandra-data-e created
persistentvolume/cassandra-data-f created
trigueros@fedora:~/multi-dc-c7a-k8s$
```

9. Despliegue y verificación de StatefulSets

Antes de desplegar StatefulSets debemos modificar los archivos statefulset-a.yaml a como se ve a continuación. Lo mismo hacemos para el statefulset-b.yaml.

```
trigueros@fedora:~/multi-dc-c7a-k8s — nano statefulset-a.yaml
~/multi-dc-c7a-k8s
GNU nano 8.3 statefulset-a.yaml
---
apiVersion: "apps/v1" #anteriormente apiVersion: "apps/v1beta1"
kind: StatefulSet
metadata:
  name: cassandraa
spec:
  serviceName: cassandraa
  replicas: 1
  selector: #agregamos este elemento selector
    matchLabels:
      app: cassandra
      interface: cassandraa
  template:
    metadata:
      labels:
        app: cassandra
        interface: cassandraa
```

```
trigueros@fedora:~/multi-dc-c7a-k8s — nano statefulset-b.yaml
~/multi-dc-c7a-k8s
GNU nano 8.3 statefulset-b.yaml
---
apiVersion: "apps/v1beta1"
kind: StatefulSet
metadata:
  name: cassandrab
spec:
  serviceName: cassandrab
  replicas: 1
  selector:
    matchLabels:
      app: cassandra
      interface: cassandrab
  template:
    metadata:
      labels:
        app: cassandra
        interface: cassandrab
[ Read 69 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
sudo kubectl -n c7a create -f statefulset-a.yaml
```

```
sudo kubectl -n c7a get pods
```

Creamos el stateful set para dc1 y comprobamos que aparece `cassandraa-0` en estado *ContainerCreating* y luego *Running*

```
trigueros@fedora:~/multi-dc-c7a-k8s
~/multi-dc-c7a-k8s

trigueros@fedora:~/multi-dc-c7a-k8s$ sudo kubectl -n c7a create -f statefulset-a.yaml
statefulset.apps/cassandraa created
trigueros@fedora:~/multi-dc-c7a-k8s$ sudo kubectl -n c7a get pods

Broadcast message from systemd-journald@fedora (Tue 2025-07-01 17:43:11 C
ST):

systemd[177568]: Failed to allocate manager object: Too many open files

NAME          READY   STATUS    RESTARTS   AGE
cassandraa-0   1/1     Running   0           31s
trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a get pods
NAME          READY   STATUS    RESTARTS   AGE
cassandraa-0   1/1     Running   0           44s
trigueros@fedora:~/multi-dc-c7a-k8s$
```

```
sudo kubectl -n c7a describe pod cassandraa-0
```

Inspecciona eventos y montajes de volúmenes

```
trigueros@fedora:~/multi-dc-c7a-k8s
~/multi-dc-c7a-k8s

Conditions:
  Type                      Status
  PodReadyToStartContainers  True
  Initialized                 True
  Ready                      True
  ContainersReady            True
  PodScheduled               True

Volumes:
  cassandra-data:
    Type:          PersistentVolumeClaim (a reference to a PersistentVolumeC
    ClaimName:     cassandra-data-cassandraa-0
    ReadOnly:      false
  kube-api-access-9lksr:
    Type:          Projected (a volume that contains injected d
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt
    Optional:      false
    DownwardAPI:   true
QoS Class:                   BestEffort
Node-Selectors:              <none>
```

`sudo kubectl -n c7a create -f statefulset-b.yaml`

Despliega `cassandrab-0`

```
trigueros@fedora:~/multi-dc-c7a-k8s
~/multi-dc-c7a-k8s

trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a create -f statefulset
-b.yaml
statefulset.apps/cassandrab created
trigueros@fedora:~/multi-dc-c7a-k8s$
```

`sudo kubectl -n c7a get statefulsets`

`sudo kubectl -n c7a get pods -o wide`

Verifica que `cassandraa-0` y `cassandrab-0` estén *Running* en nodos distintos:

```
trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a get statefulsets
NAME          READY   AGE
cassandraa    1/1     12m
cassandrab    1/1     3m12s
trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP          NODE
NOMINATED NODE READINESS GATES
cassandraa-0  1/1     Running   0           12m   10.42.3.4   nodea3
<none>         <none>
cassandrab-0  1/1     Running   0           3m18s 10.42.5.3   nodeb2
<none>         <none>
trigueros@fedora:~/multi-dc-c7a-k8s$
```

```
sudo kubectl -n c7a exec -ti cassandraa-0 -- nodetool status
```

Verificamos los nodos seed con nodetool, vemos un nodo en DC1 y otro en DC2 con estado UN (Up/Normal)

```
trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a exec -ti cassandraa-0
-- nodetool status
Datacenter: DC1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens         Owns (effective)  Host ID
Rack
UN  10.42.3.4    70.86 KiB    256             100.0%             0538493e-da98-4c
d1-8101-949a33dc9389 Rack1
Datacenter: DC2
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens         Owns (effective)  Host ID
Rack
UN  10.42.5.3    70.9 KiB     256             100.0%             0f3484aa-001c-41
88-b4a9-164d65881688 Rack1
trigueros@fedora:~/multi-dc-c7a-k8s$
```

10. Escalado del clúster

```
kubectl -n c7a scale --replicas=3 statefulset/cassandraa
```

```
kubectl -n c7a scale --replicas=3 statefulset/cassandrab
```

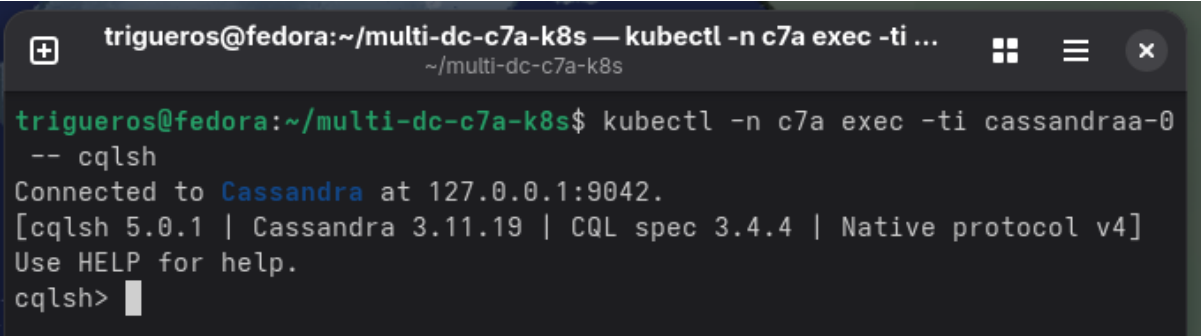
```
kubectl -n c7a get pods -o wide
```

Crea tres réplicas en cada data center y verifica que las seis pods adicionales se unan al clúster

11. Creación de keyspace y tablas en Cassandra

Entrar al shell CQL:

```
sudo kubectl -n c7a exec -ti cassandraa-0 -- cqlsh
```

A terminal window titled 'trigueros@fedora:~/multi-dc-c7a-k8s — kubectl -n c7a exec -ti ...' with a subtitle '~ /multi-dc-c7a-k8s'. The terminal shows the command 'kubectl -n c7a exec -ti cassandraa-0 -- cqlsh' being executed. The output shows a connection to Cassandra at 127.0.0.1:9042, with version information: '[cqlsh 5.0.1 | Cassandra 3.11.19 | CQL spec 3.4.4 | Native protocol v4]'. The prompt 'cqlsh>' is visible at the bottom.

```
trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a exec -ti cassandraa-0 -- cqlsh
Connected to Cassandra at 127.0.0.1:9042.
[cqlsh 5.0.1 | Cassandra 3.11.19 | CQL spec 3.4.4 | Native protocol v4]
Use HELP for help.
cqlsh>
```

Crea keyspace con NetworkTopologyStrategy

```
CREATE KEYSPACE hr_keyspace
```

```
WITH replication = {
    'class' : 'NetworkTopologyStrategy',
    'DC1': 2,
    'DC2': 2
};
```

Usa keyspace y crear tabla

```
USE hr_keyspace;
```

```
CREATE TABLE employee (
    emp_id int PRIMARY KEY,
    emp_name text,
```

```
emp_city text,  
  
emp_sal varint,  
  
emp_phone varint  
  
);
```

Insertar datos

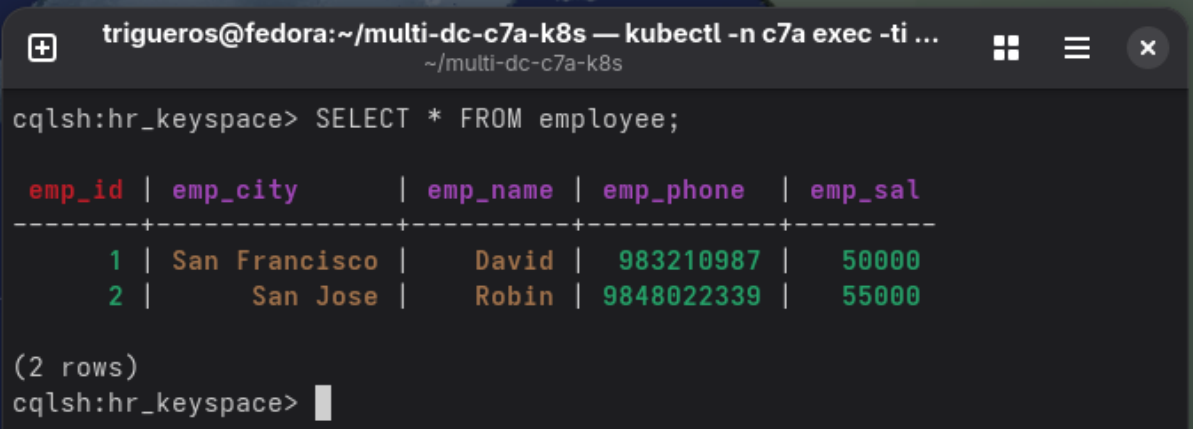
```
CONSISTENCY LOCAL_QUORUM;
```

```
INSERT INTO employee VALUES (1, 'David', 'San  
Francisco', 50000, 983210987);
```

```
INSERT INTO employee VALUES (2, 'Robin', 'San  
Jose', 55000, 9848022339);
```

Verificamos que los datos estén en ambos data centers localmente

```
SELECT * FROM employee;
```

A terminal window titled 'trigueros@fedora:~/multi-dc-c7a-k8s — kubectl -n c7a exec -ti ...' shows a Cassandra query execution. The prompt is 'cqlsh:hr_keyspace>'. The command entered is 'SELECT * FROM employee;'. The output is a table with 5 columns: emp_id, emp_city, emp_name, emp_phone, and emp_sal. There are 2 rows of data. The first row shows emp_id 1, San Francisco, David, 983210987, and 50000. The second row shows emp_id 2, San Jose, Robin, 9848022339, and 55000. Below the table, it says '(2 rows)' and the prompt 'cqlsh:hr_keyspace>' is shown again.

emp_id	emp_city	emp_name	emp_phone	emp_sal
1	San Francisco	David	983210987	50000
2	San Jose	Robin	9848022339	55000

(2 rows)
cqlsh:hr_keyspace>

12. Simulación de falla de un data center

Eliminar StatefulSet de DC1

```
sudo kubectl -n c7a delete statefulset cassandraa
```

```
sudo kubectl -n c7a get pods -o wide
```



```
trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP          NODE
NOMINATED NODE      READINESS GATES
cassandrab-0        1/1     Running   0           27m   10.42.5.3   nodeb2
<none>              <none>
cassandrab-1        1/1     Running   0           19m   10.42.6.4   nodeb3
<none>              <none>
cassandrab-2        1/1     Running   3 (17m ago) 18m   10.42.4.4   nodeb1
<none>              <none>
trigueros@fedora:~/multi-dc-c7a-k8s$
```

Conectar a DC2 y consultar

```
sudo kubectl -n c7a exec -ti cassandrab-1 -- cqlsh -e "SELECT
* FROM hr_keyspace.employee;"
```

```
trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP          NODE
NOMINATED NODE      READINESS GATES
cassandrab-0        1/1     Running   0           27m   10.42.5.3   nodeb2
<none>              <none>
cassandrab-1        1/1     Running   0           19m   10.42.6.4   nodeb3
<none>              <none>
cassandrab-2        1/1     Running   3 (17m ago) 18m   10.42.4.4   nodeb1
<none>              <none>
trigueros@fedora:~/multi-dc-c7a-k8s$ kubectl -n c7a exec -ti cassandrab-1
-- cqlsh -e "SELECT * FROM hr_keyspace.employee;"

 emp_id | emp_city          | emp_name | emp_phone | emp_sal
-----+-----+-----+-----+-----
      1 | San Francisco    | David   | 983210987 | 50000
      2 | San Jose         | Robin   | 9848022339 | 55000

(2 rows)
trigueros@fedora:~/multi-dc-c7a-k8s$
```

Vemos los mismos datos disponibles en DC2, demostrando tolerancia a fallos

13. Limpieza del entorno

```
sudo kubectl -n c7a delete statefulset,pvc,pv,svc -l
app=cassandra
```

```
sudo kubectl delete namespace c7a
```

```
k3d cluster delete multi-node-cluster
```

Elimina todos los recursos y el clúster k3d, dejando el Fedora limpio para futuros laboratorios

Referencias

ChatGPT (conversación específica)

OpenAI. (2025, junio 27). *ChatGPT conversation with Brandon Jesús Trigueros Lara* [Large language model]. Retrieved from

<https://chatgpt.com/g/g-p-679175f483a48191921eff8e16ed8bf5-bt/c/685ea496-2ea8-8010-9475-9f692193c3d8>

Tutorial de IBM Developer

IBM Developer. (2017). *10 steps to set up a multi-data center Cassandra cluster on a Kubernetes platform* [Tutorial]. Retrieved June 27, 2025, from

<https://developer.ibm.com/tutorials/ba-multi-data-center-cassandra-cluster-kubernetes-platform/>